The packed mask format (.npz)

remote_sam3 · segment.py --mask-format npz · maskio.py

In one sentence: instead of one uncompressed file per mask, each photo gets **one small file holding all of its masks**, stored at one bit per pixel and then compressed. Nothing is lost: loading the file gives back exactly the same masks.

Why the old format was so big

The old format wrote one `.npy` per mask. Each was the whole image frame as `uint8`: one **byte** for every pixel, holding only a 0 or a 1, with no compression. For a 500 × 375 photo that is 500 × 375 + a 128-byte header = **187,628 bytes per mask**, no matter how small the flower. The taxon 62741 run measured exactly that as its median mask size, and 23.1 GB for 126,653 masks.

What the new format does

Step	What happens	Size of one 500 × 375 photo with 3 masks
0. Old format	3 separate files, 1 byte per pixel	562,884 bytes
1. Stack	The photo's N masks become one N × H × W array, in one file	same bytes, 1 file instead of 3
2. Pack bits	A pixel is only 0 or 1, so 8 pixels fit in one byte (`np.packbits`)	70,313 bytes (8× smaller)
3. Compress	A mask is long runs of 0s and 1s, which deflate squeezes hard (`np.savez_compressed`)	about 2,000 bytes (measured, see below)

What is inside the file

One file per photo, named `<Genus_species_photoid>_masks.npz`, in the same `masks/batch_NNNNN/` folders as before. It is a standard NumPy archive with three arrays:

Array	Type	Contents
packed	uint8	All N masks, flattened, 8 pixels per byte
shape	int32	[N, H, W] - needed to unpack back to N masks of H × W
scores	float32	[N] - each mask's SAM3 confidence, most confident first

Masks are in the same order as before (most confident first), so mask *i* here is what used to be `_instance_i.npy`. The results CSV's `mask_paths` column holds this one path, and `mask_bytes` is this file's size. A photo with no detection gets no file, as before.

How to read it

```
from maskio import load_masks

masks, scores = load_masks('masks/batch_00001/Rudbeckia_hirta_13098033_masks.npz')
masks.shape    # (N, H, W), dtype bool   - masks[0] is the most confident
scores         # array of N confidences
```

```
# or with nothing but NumPy:
import numpy as np
d = np.load(path)
masks = np.unpackbits(d['packed'], count=int(np.prod(d['shape'])))
masks = masks.reshape(d['shape']).astype(bool)
```

What it costs and what it saves

- **Lossless.** Tested: the arrays loaded back are identical to the ones saved.
- **About 100× or more smaller.** Measured on real SAM3 masks from the earlier petal run, scaled to 500 px, 7 masks per photo: about 2 KB per photo against 1.17 MB as .npy (roughly 550×). Larger or more ragged masks compress less; expect 100-500×.
- **Fewer files.** One per photo instead of one per mask (about a third as many), which is easier on /blue and faster to transfer.
- **Slightly more work to open.** A plain `np.load` no longer returns the mask directly; use `load_masks` or the three NumPy lines above.
- **Both formats still work.** render.py, review_figure.py and the metrics read either one, chosen by the file extension. The existing taxon 62741 run stays as .npy; new runs opt in with `MASK_FORMAT=npz`, which the all-genera job sets.